# Go Programming

Typed code transcribed from the supplied screenshots

The listings below are formatted as text code, not screenshots. Small syntax issues have been corrected where needed so the examples are runnable.

## 1. HCF and LCM of Two Numbers

**main.go**

```
package main

import "fmt"

func main() {
    var a, b int
    fmt.Print("Enter a and b: ")
    fmt.Scan(&a, &b)

    x, y := a, b
    for y != 0 {
        x, y = y, x%y
    }
    gcd := x

    if gcd == 0 {
        fmt.Println("HCF and LCM are undefined for both inputs being zero.")
        return
    }

    lcm := (a / gcd) * b
    if lcm < 0 {
        lcm = -lcm
    }
    if gcd < 0 {
        gcd = -gcd
    }

    fmt.Println("LCM and HCF are:", lcm, gcd)
}
```

## 2. Number Pattern (Increasing numbers in each row)

**main.go**

```
package main

import "fmt"

func main() {
    var n int
    fmt.Print("Enter the value of N: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        for j := 0; j < n-i-1; j++ {
            fmt.Print(" ")
        }
        for j := 0; j <= i; j++ {
            fmt.Print(j + 1)
            fmt.Print(" ")
        }
        fmt.Println()
    }
}
```

## 3. Student Structure and Display Method

**student/student.go**

```
package student

import "fmt"

type Student struct {
    Name   string
    Rollno int
    Marks  int
}

func (s Student) Display() {
    fmt.Println("Name:", s.Name)
    fmt.Println("Rollno:", s.Rollno)
    fmt.Println("Marks:", s.Marks)
}
```

**main.go (module name: structdemo)**

```
package main

import "structdemo/student"

func main() {
    s := student.Student{
        Name:   "Krupakar",
        Rollno: 101,
        Marks:  98,
    }
    s.Display()
}
```

## 4. Mean and Standard Deviation

**main.go**

```
package main

import (
    "fmt"
    "math"
)

func main() {
    var n int
    fmt.Print("Enter N: ")
    fmt.Scan(&n)
    if n <= 0 {
        fmt.Println("N must be greater than zero.")
        return
    }

    numbers := make([]float64, n)
    sum := 0.0
    fmt.Println("Enter values:")
    for i := 0; i < n; i++ {
        fmt.Scan(&numbers[i])
        sum += numbers[i]
    }

    mean := sum / float64(n)
    variance := 0.0
    for i := 0; i < n; i++ {
        diff := numbers[i] - mean
        variance += diff * diff
    }
    variance /= float64(n)

    sd := math.Sqrt(variance)
    fmt.Println("Mean and SD are:", mean, sd)
}
```

## 5. Number Pattern (Repeated sequence)

**main.go**

```
package main

import "fmt"

func main() {
    var n int
    fmt.Print("Enter N: ")
    fmt.Scan(&n)

    x := 1
    for i := 0; i < n; i++ {
        for j := 0; j <= i; j++ {
            fmt.Print(x, " ")
            x++
        }
        fmt.Println()
    }
}
```

## 6. Concatenate Two Strings

**main.go**

```
package main

import "fmt"

func main() {
    var s1, s2 string
    fmt.Print("Enter two words: ")
    fmt.Scan(&s1, &s2)

    s := s1 + s2
    fmt.Println(s)
}
```

## 7. Check Whether a String Is a Palindrome

**main.go**

```
package main

import "fmt"

func main() {
    var s string
    fmt.Print("Enter a string: ")
    fmt.Scan(&s)

    reverse := ""
    for i := len(s) - 1; i >= 0; i-- {
        reverse += string(s[i])
    }

    if s == reverse {
        fmt.Println("Same")
    } else {
        fmt.Println("Not")
    }
}
```

## 8. Contact Structure and Input/Display

**main.go**

```
package main

import "fmt"

type Contact struct {
    Name  string
    Age   int
    Phone string
    Email string
}

func main() {
    var person Contact

    fmt.Print("Enter person name: ")
    fmt.Scan(&person.Name)
    fmt.Print("Enter person age: ")
    fmt.Scan(&person.Age)
    fmt.Print("Enter person phone number: ")
    fmt.Scan(&person.Phone)
    fmt.Print("Enter person email: ")
    fmt.Scan(&person.Email)

    fmt.Println("Person Name:", person.Name)
    fmt.Println("Person Age:", person.Age)
    fmt.Println("Person Phone:", person.Phone)
    fmt.Println("Person Email:", person.Email)
}
```

## 9. Average of N Integers

**main.go**

```
package main

import "fmt"

func main() {
    var n int
    fmt.Print("Enter N: ")
    fmt.Scan(&n)
    if n <= 0 {
        fmt.Println("N must be greater than zero.")
        return
    }

    numbers := make([]int, n)
    sum := 0
    for i := 0; i < n; i++ {
        fmt.Scan(&numbers[i])
        sum += numbers[i]
    }

    fmt.Println("Average is:", float64(sum)/float64(n))
}
```

## 10. Remove Duplicate Integers from a Slice

**main.go**

```
package main

import "fmt"

func removeDuplicates(arr []int) []int {
    seen := make(map[int]bool)
    result := make([]int, 0, len(arr))

    for _, value := range arr {
        if seen[value] {
            continue
        }
        seen[value] = true
        result = append(result, value)
    }
    return result
}

func main() {
    var n int
    fmt.Print("Enter size: ")
    fmt.Scan(&n)
    if n < 0 {
        fmt.Println("Size cannot be negative.")
        return
    }

    arr := make([]int, n)
    fmt.Println("Enter", n, "numbers:")
    for i := 0; i < n; i++ {
        fmt.Scan(&arr[i])
    }

    fmt.Println("Result:", removeDuplicates(arr))
}
```

## 11. Reverse-Sort Integer and String Slices

**main.go**

```
package main

import (
    "fmt"
    "sort"
)

func reverseSortedIntegers(arr []int) []int {
    result := append([]int(nil), arr...)
    sort.Sort(sort.Reverse(sort.IntSlice(result)))
    return result
}

func reverseSortedStrings(arr []string) []string {
    result := append([]string(nil), arr...)
    sort.Sort(sort.Reverse(sort.StringSlice(result)))
    return result
}

func main() {
    numbers := []int{10, 5, 30, 20, 15}
    words := []string{"Apple", "Mango", "Banana", "Orange"}

    fmt.Println("Original integer array:", numbers)
    fmt.Println("Reverse sorted integers:", reverseSortedIntegers(numbers))
    fmt.Println("Original string array:", words)
    fmt.Println("Reverse sorted strings:", reverseSortedStrings(words))
}
```

## 12. String Package Functions

**main.go**

```
package main

import (
    "fmt"
    "strings"
)

func main() {
    str := "Hello Golang"

    fmt.Println("Contains:", strings.Contains(str, "Go"))
    fmt.Println("ContainsAny:", strings.ContainsAny(str, "xyz"))
    fmt.Println("Count:", strings.Count(str, "l"))
    fmt.Println("EqualFold:", strings.EqualFold("Golang", "golang"))
}
```